Singleton

One MetricsRegistry across request threads

Season 1 · GoF · Beginner

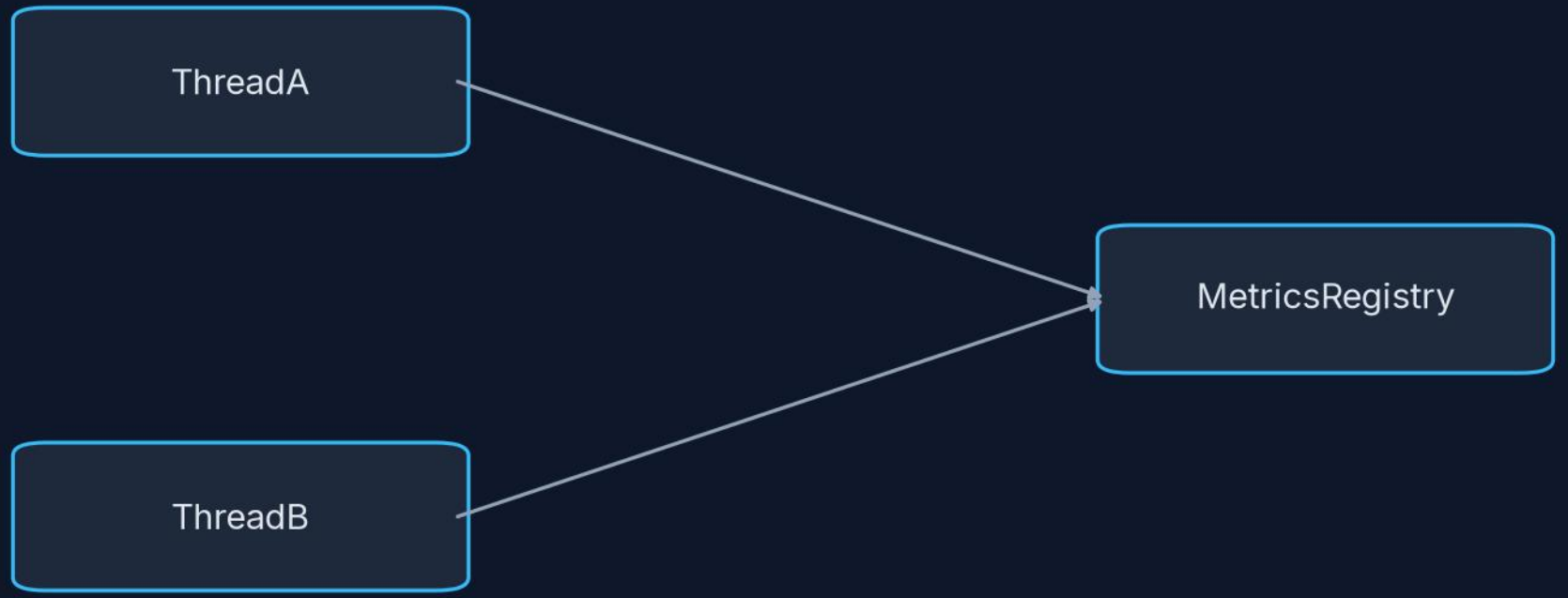
Today's Hook

- Problem: One MetricsRegistry across request threads
- Naive pain cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- We will refactor to Singleton with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Scenario: One MetricsRegistry across request threads
- Constraints: solve «One MetricsRegistry across request threads» with Singleton — keep the demo runnable and the failure path observable.
- Naive pain: Naive: each service constructs its own counter — totals disagree.
- Success looks like: naiveA=2 naiveB=1

Structure



Singleton: One MetricsRegistry across request threads

Real-World Scenario

- Running example: One MetricsRegistry across request threads
- Key types: singleton, AppConfig
- Happy-path stdout cue: naiveA=2 naiveB=1
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.gof.creational.singleton.problem;

/** Naive: each service constructs its own counter — totals disagree. */
public final class NaiveMetrics {
    private long hits;
    public void hit() { hits++; }
    public long hits() { return hits; }
}
```

Why Naive Hurts

- Naive: each service constructs its own counter — totals disagree.
- Without Singleton, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.gof.creational.singleton;

/** Preferred: enum singleton — serialization-safe, reflection-safe. */
public enum AppConfig {
    INSTANCE;

    private String env = "prod";

    public String env() { return env; }
    public void setEnv(String env) { this.env = env; }
    public String greeting() { return "hello from " + env; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.creational.singleton.problem;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Singleton: one MetricsRegistry ===");
        var a = new NaiveMetrics(); var b = new NaiveMetrics();
        a.hit(); a.hit(); b.hit();
        System.out.println("naiveA=" + a.hits() + " naiveB=" + b.hits());
        var reg = com.buildwithwaqar.patterns.gof.creational.singleton.MetricsRegistry.getInstance();
        reg.reset(); reg.hit(); reg.hit(); reg.hit();
        System.out.println("singletonHits=" + reg.hits());
        System.out.println("same=" + (reg == com.buildwithwaqar.patterns.gof.creational.singleton.MetricsRegistry.getInstance()));

    }
}
```

Walkthrough Output

```
=== Singleton: one MetricsRegistry ===
naiveA=2 naiveB=1
singletonHits=3
same=true
```

Step-by-Step Trace

- Match stdout to stages — look for: naiveA=2 naiveB=1
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp ... 03-singleton ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Singleton
// Prefer records + sealed where they shrink the demo.
// Problem: One MetricsRegistry across request threads
```

Trade-offs

- Use Singleton when «One MetricsRegistry across request threads» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Singleton in product language.

Related Patterns

- Name the pattern usually confused with Singleton and separate the forces.
- Sketch the roles for «One MetricsRegistry across request threads» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: One MetricsRegistry across request threads
- Naive pain → Singleton boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.